



Embarrassingly Simple Self-Distillation Improves Code Generation

Ruixiang Zhang* Richard He Bai* Huangjie Zheng* Navdeep Jaitly Ronan Collobert
Yizhe Zhang*

Apple

<https://github.com/apple/ml-ssd>

Can a large language model (LLM) improve at code generation using only its own raw outputs, without a verifier, a teacher model, or reinforcement learning? We answer in the affirmative with simple self-distillation (SSD): sample solutions from the model with certain temperature and truncation configurations, then fine-tune on those samples with standard supervised fine-tuning. SSD improves Qwen3-30B-Instruct from 42.4% to 55.3% pass@1 on LiveCodeBench v6, with gains concentrating on harder problems, and it generalizes across Qwen and Llama models at 4B, 8B, and 30B scale, including both instruct and thinking variants. To understand why such a simple method can work, we trace these gains to a *precision-exploration conflict* in LLM decoding and show that SSD reshapes token distributions in a context-dependent way, suppressing distractor tails where precision matters while preserving useful diversity where exploration matters. Taken together, SSD offers a complementary post-training direction for improving LLM code generation.

Correspondence: {yizhe.zhang, ruixiang.zhang, richard.bai, huangjie.zheng}@apple.com

Date: April 2, 2026 * Equal contribution

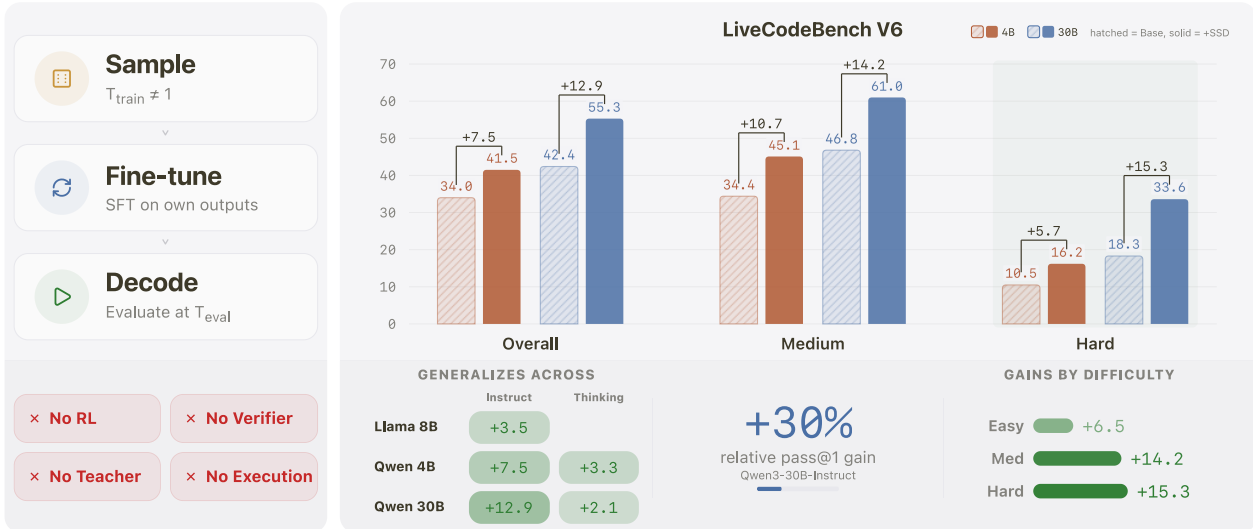


Figure 1 Simple self-distillation (SSD) is embarrassingly simple, yet yields substantial LiveCodeBench v6 gains across five models spanning two families, three scales, with both instruct and thinking variants. Left: SSD samples from the base model with training-time decoding temperature T_{train} , fine-tunes on its own raw outputs, and then decodes at evaluation time with T_{eval} ; it uses no RL, verifier, teacher, or code execution environment. Right: LiveCodeBench v6 pass@1 for Qwen3-4B-Instruct and Qwen3-30B-Instruct on the Overall, Medium, and Hard splits (orange = 4B, blue = 30B; hatched = base, solid = +SSD). The footer highlights the broader pattern: all five evaluated models improve, Qwen3-30B-Instruct gains +30% relative pass@1, and the largest gains occur on harder problems.

Table 1 Comparison of training paradigms.

Method	Dense Signal	No Teacher	No Verifier	No Privileged Info
SFT on External Data	✓	✗	✓	✓
GRPO	✗	✓	✗	✓
On-Policy Distillation	✓	✗	✓	✓
On-Policy Self-Distillation	✓	✓	✓	✗
Simple Self-Distillation (SSD) Ours	✓	✓	✓	✓

1 Introduction

As LLMs are deployed to increasingly difficult coding tasks, the supply of high-quality supervised signal has become a binding constraint: human-written solutions (Chen et al., 2021; Austin et al., 2021; Hendrycks et al., 2021a) are expensive to produce, and synthetic data pipelines require either a stronger teacher model (Hinton et al., 2015; Kim and Rush, 2016; Hsieh et al., 2023; Agarwal et al., 2024) or execution-based verification for every training problem (Li et al., 2022; Le et al., 2022; Singh et al., 2024; Liu et al., 2025). Teacher-based distillation also inherits the ceiling of the teacher, while reinforcement learning with verifiable reward remains operationally complex and can be unstable, even in recent RL-based reasoning and coding pipelines (He et al., 2026; Shao et al., 2024; DeepSeek-AI, 2025; OpenAI, 2025). Unsupervised alternatives that use intrinsic rewards such as majority voting or entropy minimization (Zuo et al., 2025; Agarwal et al., 2025) have shown early promise but face reward hacking and collapse under extended training (Zhang et al., 2025). This raises a natural question: can a model improve itself without leveraging any external labeled data or verification at all?

We show the answer is yes. Our method, *simple self-distillation* (SSD), is embarrassingly simple: sample solutions from the base model with specified temperature and truncation, then fine-tune on those raw, unverified samples via standard cross-entropy loss. This method requires only a set of problem prompts and the model itself: no human-labeled solutions, no reference answers, no teacher model, no reward model, no verifier, no execution environment, and no reinforcement learning of any kind.

Surprisingly, it works. SSD improves Qwen3-30B-Instruct from 42.4% to 55.3% pass@1 on LiveCodeBench v6 (Jain et al., 2024), with especially large gains on hard problems. Coverage improvements are larger still: hard-problem pass@5 rises from 31.1% to 54.1%, suggesting that SSD preserves useful exploration across solution branches instead of only sharpening a single dominant mode. These gains are not model-specific: SSD generalizes across five models spanning two families, three scales, and both instruct and thinking models.

We study why such a simple method can work in the domain of code generation, which serves as a particularly useful testbed because the task structure makes the underlying mechanism especially visible. Code interleaves *fork* positions, where several continuations are genuinely plausible and may correspond to different solution approaches, with *lock* positions, where syntax and semantics leave little ambiguity but a low-probability distractor tail still remains. These two context types make contradictory demands on decoding temperature T_{eval} . Lowering T_{eval} secures locks but starves forks of diversity, while raising it enables exploration at forks but lets distractors flood back in at locks. The best global decoding setting is therefore necessarily a compromise; we call this tension the *precision-exploration conflict*.

SSD can be understood through this lens. Training on temperature-shifted, truncated samples implicitly reshapes the model’s distributions in a context dependent way (we formalize this later as *support compression* and *within support reshaping* in Equation (4)): it suppresses distractors most aggressively at locks while preserving useful diversity at forks. Evidence from controlled simulation and real-model analysis (Section 4.2), together with theoretical analysis (Section B), supports this account and explains why changing only the decoding settings cannot recover the gains. More broadly, this suggests that existing code models contain capability not realized under fixed decoding alone.

Our contributions are threefold. First, we show that SSD can substantially improve code generation models’ per-

formance using only their own unverified outputs, without any external teacher, verifier, reward model, or labeled solutions. Second, we identify the *precision-exploration conflict* and argue that it is the key mechanism behind SSD. Third, we support this mechanism with aligned evidence from controlled simulation, real-model analysis, and theory.

2 Embarrassingly Simple Self-Distillation (SSD)

Data synthesis. We write T for the temperature and ρ for the truncation configuration, namely the top- k and top- p used in decoding (see [Section A](#) for the precise decoding procedure). Given a *frozen* pre-trained LLM p_θ and a set of prompts $\mathcal{X}$, we sample N candidate solutions per problem:

$$y \sim \text{Decode}_{T_{\text{train}}, \rho_{\text{train}}}[p_\theta(\cdot \mid x)] \quad (1)$$

The solutions are not verified in any way: no execution, no test cases, no filtering by correctness. The raw outputs form the simple self-distillation dataset $\mathcal{D}_{\text{SSD}}$. In practice, $N=1$ (a single sample per prompt) already suffices ([Section 3](#)).

Training. We fine-tune the model on $\mathcal{D}_{\text{SSD}}$ with standard supervised fine-tuning (SFT):

$$\mathcal{L}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SSD}}} \sum_{t=1}^{|y|} \log p_\theta(y_t \mid x, y_{<t}) \quad (2)$$

Inference. The fine-tuned model p_{θ^*} is deployed with evaluation-time decoding configuration $(T_{\text{eval}}, \rho_{\text{eval}})$:

$$\hat{y} \sim \text{Decode}_{T_{\text{eval}}, \rho_{\text{eval}}}[p_{\theta^*}(\cdot \mid x)] \quad (3)$$

3 Experiments

3.1 Experimental Setup

Models. We evaluate SSD on five models spanning two families (Llama, Qwen), three scales (4B–30B), and two reasoning styles (instruct, thinking): Llama-3.1-8B-Instruct (dense, 8B), Qwen3-4B-Instruct-2507 (dense, 4B; hereafter *Qwen3-4B-Instruct*), Qwen3-4B-Thinking-2507 (dense, 4B; hereafter *Qwen3-4B-Thinking*), Qwen3-30B-A3B-Instruct-2507 (MoE, 30B total / 3B active; hereafter *Qwen3-30B-Instruct*), and Qwen3-30B-A3B-Thinking-2507 (MoE, 30B / 3B active; hereafter *Qwen3-30B-Thinking*). We apply SSD to each of these *base* models.

Data synthesis. We use the seed subset of the rSTARcoder dataset ([Liu et al., 2025](#)), de-duplicated to yield $\sim 10\text{K}$ unique competitive programming problems. For each prompt we sample a single solution from the base model using the per-model generation decoding configuration in [Table 3](#). We apply only minimal syntactic filtering to remove empty responses and single line stubs, meaning there is absolutely **no correctness signal** used. Generation uses vLLM ([Kwon et al., 2023](#)) with 128K maximum sequence length limit.

Training. We fine-tune with Megatron-LM¹ on $8 \times \text{B200}$ GPUs (EP=8 for MoE models), using AdamW with cosine decay (peak LR 5×10^{-6}), global batch size 32, sequence length 65,536, and 2,500 iterations for instruct models and 300 iterations for thinking models. The learning rate is warmed-up with 250 and 50 iterations, respectively.

Evaluation. Our primary benchmark is LiveCodeBench v6 (LCB v6; Feb–May 2025, following the version split adopted by recent model releases²; stratified by easy/medium/hard). We report LCB v5 (Aug 2024–Feb 2025, following the version split adopted by rSTARcoder ([Liu et al., 2025](#))) as a secondary confirmation. The primary metric is pass@1; we also report pass@5 and per-difficulty breakdowns. Base-model results use each model’s officially recommended sampling parameters ([Table 4](#)); SSD models are evaluated with the decoding settings in [Table 3](#). Full experimental details, prompt formatting, and decoding settings are given in [Section C.1](#).

Table 2 SSD improves every evaluated model on LiveCodeBench, with the largest gains on medium and hard problems. Results on LCB v6 and LCB v5, broken down by difficulty and grouped by reasoning style (thinking vs. instruct). Within each model pair, the first row is the base model and the second is +SSD; cell shading encodes the change relative to the base row (green = improvement, red = decrease).

LiveCodeBench v6 02/2025 – 05/2025									LiveCodeBench v5 08/2024 – 02/2025								
	PASS@1				PASS@5					PASS@1				PASS@5			
	ALL	EASY	MED	HARD	ALL	EASY	MED	HARD		ALL	EASY	MED	HARD	ALL	EASY	MED	HARD
THINKING																	
Qwen3-4B-Thinking	54.5	97.4	59.2	29.7	67.5	100.0	75.4	45.8	59.6	98.0	69.5	31.4	70.3	98.9	80.1	47.5	
+ SSD	57.8 +3.3	98.7 +1.3	62.8 +3.6	33.8 +4.1	71.4 +3.9	100.0	77.3 +1.9	53.1 +7.3	63.1 +3.5	98.5 +0.5	74.1 +4.6	35.7 +4.3	74.7 +4.4	98.9	86.8 +6.7	52.8 +5.3	
Qwen3-30B-Thinking	66.1	99.8	73.0	41.5	76.7	100.0	84.7	59.7	69.1	98.6	81.7	43.9	79.8	98.9	91.4	61.1	
+ SSD	68.2 +2.1	100.0 +0.2	76.4 +3.4	46.7 +5.2	80.2 +3.5	100.0	87.1 +2.4	65.8 +6.1	69.9 +0.8	98.4 -0.2	82.9 +1.2	44.9 +1.0	80.5 +0.7	98.9	91.8 +0.4	62.4 +1.3	
INSTRUCT																	
Llama-3.1-8B-Instruct	12.7	46.8	5.4	0.0	23.0	76.7	15.1	0.8	12.7	45.1	4.1	0.7	20.8	64.7	11.6	2.8	
+ SSD	16.2 +3.5	52.6 +5.8	10.0 +4.6	1.6 +1.6	24.0 +1.0	73.3 -3.4	17.1 +2.0	3.3 +2.5	13.5 +0.8	45.2 +0.1	5.9 +1.8	1.3 +0.6	21.9 +1.1	67.1 +2.4	12.3 +0.7	3.7 +0.9	
Qwen3-4B-Instruct	34.0	79.7	34.4	10.5	41.0	90.0	40.5	16.5	34.3	85.4	28.4	10.1	45.4	96.0	45.4	17.5	
+ SSD	41.5 +7.5	86.5 +6.8	45.1 +10.7	16.2 +5.7	56.8 +15.8	98.1 +8.1	59.7 +19.2	34.1 +17.6	45.7 +11.4	90.2 +4.8	52.1 +23.7	16.6 +6.5	61.9 +16.5	98.8 +2.8	73.1 +27.7	33.6 +16.1	
Qwen3-30B-Instruct	42.4	84.5	46.8	18.3	53.5	93.3	56.8	31.1	45.8	91.2	49.8	17.7	58.7	96.1	69.2	30.6	
+ SSD	55.3 +12.9	91.0 +6.5	61.0 +14.2	33.6 +15.3	71.6 +18.1	99.9 +6.6	76.4 +19.6	54.1 +23.0	54.3 +8.5	92.9 +1.7	64.1 +14.3	25.9 +8.2	70.7 +12.0	97.6 +1.5	83.0 +13.8	47.2 +16.6	

3.2 SSD Improves Code Generation Across the Board

SSD yields large gains on LiveCodeBench. On LCB v6, SSD improves Qwen3-30B-Instruct from 42.4% to 55.3% pass@1 (+12.9pp, +30.4% relative). The gains are broad across the evaluated models: Llama-8B improves by +3.5pp, Qwen3-4B-Instruct by +7.5pp, Qwen3-4B-Thinking by +3.3pp, and Qwen3-30B-Thinking by +2.1pp. Substantial gains also appear on the larger 374-problem LCB v5, where Qwen3-30B-Instruct improves from 45.8% to 54.3% pass@1 (+8.5pp). For a recipe based only on self-generated, unverified solutions and standard supervised fine-tuning, these are consistently strong improvements.

SSD helps most on harder problems. For Qwen3-30B-Instruct on LCB v6, pass@1 improves by +6.5pp on easy problems, +14.2pp on medium problems, and +15.3pp on hard problems. The same concentration appears at pass@5, where the gains are +6.6pp on easy, +19.6pp on medium, and +23.0pp on hard. A similar pattern is visible in the Qwen thinking models: for Qwen3-4B-Thinking, hard pass@1 improves by +4.1pp versus +1.3pp on easy problems, and for Qwen3-30B-Thinking, the corresponding gains are +5.2pp versus +0.2pp. Across Table 2, the medium and hard splits consistently account for the largest absolute gains.

SSD does not collapse diversity. A second clear pattern in the table is that the gains are often larger at pass@5 than at pass@1, indicating that SSD preserves and even improves generation diversity. Across the Qwen models on LCB v6, the pass@5 gain exceeds the corresponding pass@1 gain: +15.8pp versus +7.5pp for Qwen3-4B-Instruct, +3.9pp versus +3.3pp for Qwen3-4B-Thinking, +18.1pp versus +12.9pp for Qwen3-30B-Instruct, and +3.5pp versus +2.1pp for Qwen3-30B-Thinking. For Qwen3-30B-Instruct, the same asymmetry is even larger on the hard subset, where pass@5 rises by +23.0pp while pass@1 rises by +15.3pp; on LCB v5, pass@5 likewise improves more than pass@1 (+12.0pp versus +8.5pp). These larger pass@5 gains are consistent with improved diversity across generation samples.

Because SSD is trained only on competitive-programming data, a natural concern is that it may hurt performance outside that domain. To test this, we evaluate the same trained model out of the box on benchmarks for math reasoning, general code generation, and code understanding, without any additional training or adaptation. Performance remains broadly stable for the 30B models, see Section C.3.

¹<https://github.com/NVIDIA/Megatron-LM>

²<https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>



Figure 2 SSD outperforms the best point in the evaluated base-model decoding sweep within standard global decoding policies. Each panel shows one model (30B-Instruct, 4B-Instruct, 4B-Thinking) and one metric (pass@1 or pass@5); amber curves sweep base-model evaluation temperature while blue horizontal lines mark SSD results from Table 2. Solid shading marks the margin over all problems; outlined (dashed-border) shading marks the margin on hard problems.

3.3 Global Decoding Policies Cannot Match SSD

Could the gains in Table 2 be recovered *without* training, by tuning only evaluation-time decoding on the original base model? We test this within the standard family of global temperature and truncation ($T_{\text{eval}}, \rho_{\text{eval}}$) decoding policies, sweeping evaluation-time decoding settings on the base model and comparing the best evaluated base-model configuration to SSD. For this comparison, we follow the officially recommended sampling settings for each model (see Table 4) and sweep temperature extensively to probe the strongest decode-only performance achievable by the base model. Figure 2 visualizes representative temperature sweeps and the remaining gap to SSD.

Temperature tuning yields only modest gains on the base model. The base-model sweep curves are strikingly flat: for Qwen3-30B-Instruct, pass@1 ranges from 41.3% to 43.5% across the evaluated temperatures, a spread of only 2.2 pp. The other models show similarly narrow ranges (1.5–3.0 pp; Figure 2).

SSD still outperforms the best-tuned base model, especially on hard problems and at pass@5. Comparing SSD against the best-tuned base model, pass@1 advantages remain: +11.8 pp for Qwen3-30B-Instruct, +5.8 pp for Qwen3-4B-Instruct, +2.2 pp for Qwen3-4B-Thinking, and +1.1 pp for Qwen3-30B-Thinking. The gap widens on hard problems: for Qwen3-30B-Instruct, SSD exceeds the best-tuned base model by +13.3 pp on hard pass@1 and +19.4 pp on hard pass@5, both larger than the corresponding all-problem margins. This pattern holds across all models, the SSD advantage is consistently largest on hard problems at pass@5. These persistent margins indicate that SSD produces changes in the model itself in ways no decoding configuration can replicate, an effect we investigate in Section 4.

3.4 How SSD Hyperparameters Interact

To understand the best configuration of training and inference hyperparameters for SSD, we performed a grid search over T_{train} , and evaluated each checkpoint at multiple T_{eval} , with Qwen3-30B-Instruct on LCB v6. We compare two regimes: a no-truncation ablation, where temperature composition is cleanest, and the full truncated setting, where training-time truncation provides an additional gain channel. Full details are in Section C.2.

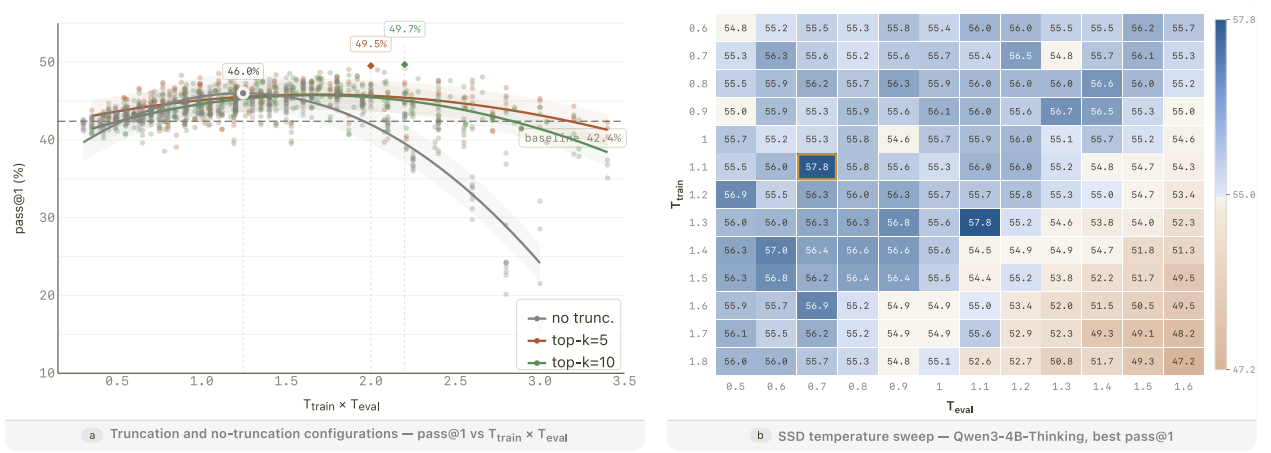


Figure 3 Training and evaluation temperatures compose through a broad effective-temperature band, while truncation raises the achievable pass@1 within that band. (a) Representative Qwen3-30B-Instruct sweeps on LCB v6 against $T_{\text{eff}} = T_{\text{train}} T_{\text{eval}}$: gray = no truncation, amber/green = truncated training-time sampling. Dots are runs, curves are quadratic fits, and the dashed line marks the 42.4% baseline. **(b)** Qwen3-4B-Thinking on LCB v6 with truncation, shown as best pass@1 across iterations over $(T_{\text{train}}, T_{\text{eval}})$.

Without truncation, effective temperature organizes performance. T_{train} and T_{eval} trade off each other: T_{train} controls how strongly SSD reshapes the model distribution, while T_{eval} controls how aggressively decoding exploits that reshaped distribution. Define $T_{\text{eff}} = T_{\text{train}} \cdot T_{\text{eval}}$; we show that T_{eff} governs performance. To isolate this, we run a search with only temperature scaling ($T_{\text{train}} \in \{0.5, 0.7, 1.0, 1.5, 2.0\}$ and $T_{\text{eval}} \in [0.6, 1.5]$; Figure 3a, Figure 10), and no truncation ρ_{train} . In this regime, the two temperatures compose cleanly: configurations are well governed by T_{eff} , with $R^2=0.75$ and a quadratic peak near $T_{\text{eff}} \approx 1.2$, as formalized in Section B.3. This also explains why higher T_{train} makes the model more responsive to T_{eval} : stronger training-time reshaping creates more room for evaluation-time decoding to trade off precision against diversity. Intuitively, higher T_{eff} is preferred to have more diverse generation as long as the generation is not broken.

With truncation, the performance ceiling rises. When a nontrivial training-time truncation configuration ρ_{train} is used during SSD data generation, the truncated runs (amber/green in Figure 3a) remain above the baseline across a wider range of T_{eff} than the no-truncation runs (gray), though the exact collapse onto T_{eff} no longer holds. This is expected: training-time truncation adds a second improvement channel on top of temperature composition by suppressing low-probability tails during data synthesis. Among the truncated runs, the best observed setting uses $T_{\text{train}}=2.0$, $T_{\text{eval}}=1.1$, and training-time top- $k=10$, reaching 49.7% pass@1 (+7.3 pp), above all no-truncation results. As expected, the optimal T_{eff} generally shifts towards a higher temperature with more stringent truncation. Similarly, the diagonal-band pattern appears for Qwen3-4B-Thinking (Figure 3b, Figure 11), confirming that the temperature-composition structure extends to thinking models.

4 Why SSD Works

The gains above raise a natural question: what changes inside the model during simple self-distillation, and why can't the same effect be achieved by simply adjusting how the original model decodes? Our hypothesis is that the answer lies in a structural conflict in generation. Some tokens demand precision, others demand exploration, and any fixed decoding configuration must compromise between them. SSD helps by reshaping token distributions in a way that alleviates this conflict. We validate this mechanism in three steps: a controlled toy simulation, real model analysis, and theoretical decomposition.

4.1 The Precision-Exploration Conflict Hypothesis

We will take code generation as an example. At certain positions, syntax and context leave almost no ambiguity: after `if n ==`, the model must produce a specific value, and it knows which one, yet a long tail of syntactically

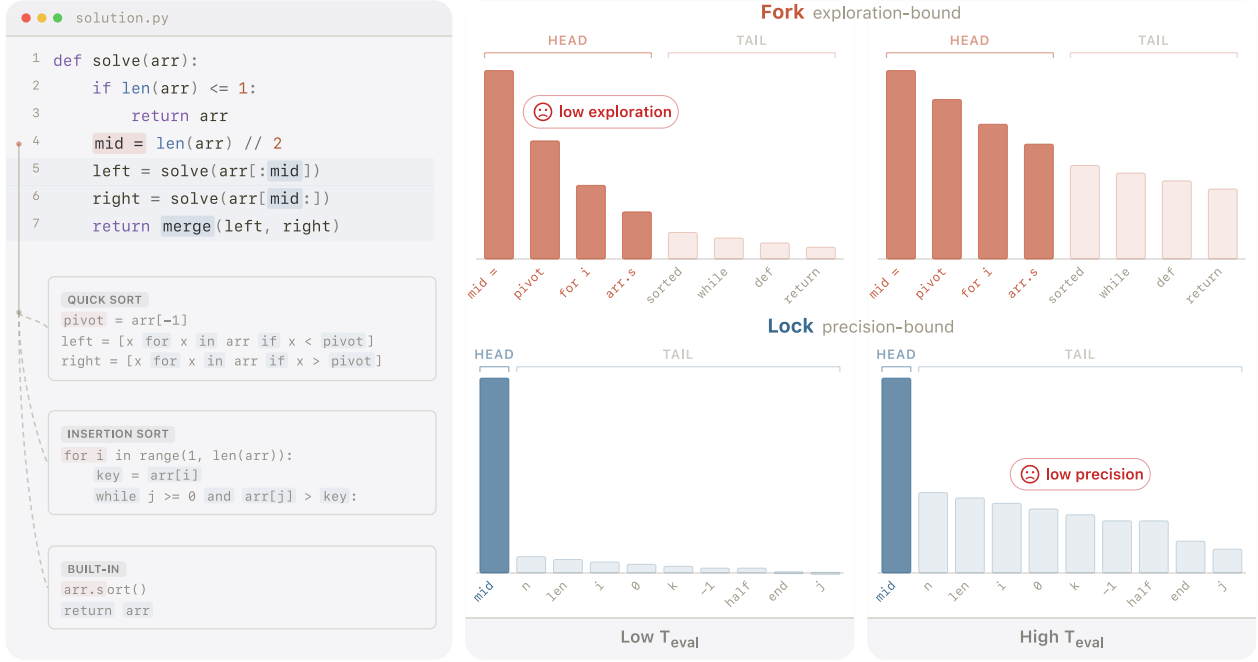


Figure 4 A single evaluation temperature cannot satisfy both exploration at forks and precision at locks. **Left:** a sorting example in which the algorithm-choice token is a *fork* position (rust-orange), while the later uses of `mid` are *lock* positions (blue); gray ghost branches indicate other valid algorithms that could have been taken at the fork. **Right:** token distributions for the same two context types under low and high T_{eval} , with head and tail mass shown explicitly. Low T_{eval} keeps the lock precise but collapses the fork’s viable head (*low exploration*); high T_{eval} restores exploration at the fork but revives the lock’s distractor tail (*low precision*).

plausible alternatives still carries nontrivial probability mass. At other positions, the distribution is genuinely spread across multiple viable continuations: when beginning the body of a function, the model might open with a `for` loop, a recursive call, or a data-structure initialization, each leading to a fundamentally different solution. We hypothesize that these two kinds of positions make fundamentally contradictory demands on the decoding configuration (Figure 4).

We call the first a **lock**: a position where the distribution is sharply peaked, with very few tokens carrying most of the mass and a long distractor tail carrying the rest. We call the second a **fork** (Bigelow et al., 2025; Wang et al., 2025b): a position where the distribution is spread across multiple plausible tokens that can lead to meaningfully different downstream continuations. Locks demand precision: commit to the dominant token and suppress the tail. Forks demand exploration: spread mass across viable alternatives to avoid missing the good paths.

Under this view, inference temperature T_{eval} is what makes the conflict irreconcilable. Scaling by T_{eval} flattens or sharpens the entire distribution $p_T(v) \propto p(v)^{1/T}$: higher T_{eval} compresses probability gaps, pulling tokens toward equal footing; lower T_{eval} widens them, amplifying the dominant peak. There is a dilemma. Lowering temperature sharpens the peak at a lock, suppressing distractors, but starves a fork of the diversity it needs. Raising temperature diversifies the head at a fork, giving lower-ranked correct continuations a chance, but destabilizes locks as the distractor tail regains mass. The best global setting, applied to every context in the sequence, is therefore necessarily a compromise: the temperature that helps forks is precisely what lets distractors resurface at locks.

If this picture is right, then SSD should not sharpen the model uniformly: it should suppress distractor tails at locks while leaving more useful room for exploration at forks.

4.2 How SSD Reshapes a Model: Toy Simulation and Real-Model Analysis

We now test that prediction in two settings. We begin with a toy environment where the conflict is explicit and success can be computed exactly. We then ask whether the same qualitative pattern appears in a real model.

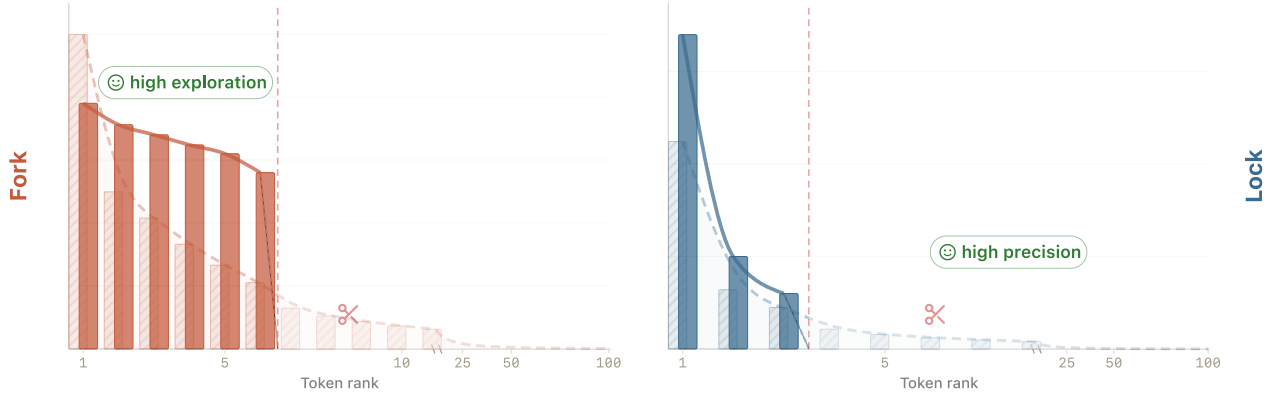


Figure 5 SSD turns forks into plateaus and locks into spikes. Tokens are ranked by probability. Hatched bars and dashed curves show the base model; solid bars and solid curves show the model after SSD; the red dashed cutoff marks the support retained during SSD. **(a)** Fork-like state: the diffuse tail is trimmed, but several top continuations remain and become more evenly weighted, forming a broad plateau over viable branches. **(b)** Lock-like state: the same rule trims the tail much more aggressively and concentrates mass on the dominant token, producing a sharper spike.

Controlled simulation. We begin with a minimal environment that contains exactly the structure in our hypothesis. Successful trajectories must pass through one fork state and then three lock states before reaching PASS; any wrong decision leads to FAIL. At the fork, several continuations are genuinely plausible. At each lock, one token is correct but a distractor tail remains. Because every transition is specified explicitly, the probability of success can be computed in closed form for any decoding temperature (full details are given in [Section C.4](#)).

Even in this minimal setting, the same dilemma appears. Sweeping a single global decoding temperature on the base model recovers the same tradeoff as in [Section 4.1](#): colder decoding protects the locks but starves the fork, while hotter decoding helps the fork but breaks the locks ([Figure 14](#)). The base model therefore operates at a narrow compromise.

In the toy, SSD changes that compromise by reshaping the two regimes differently ([Figure 5](#)). At lock-like states, the low-probability tail is stripped away, so the dominant token becomes much harder to dislodge. At fork-like states, several plausible continuations remain near the top, but the useless tail is reduced and the surviving options become more even. These local changes widen the viable decoding regime itself: after SSD, the best decoding temperature shifts much higher, and success probability rises substantially.

The synergy between training and decoding. The toy also shows why training and decoding are complementary rather than interchangeable. Training does not solve the fork by itself; it makes the locks less fragile. Decode-only temperature tuning does not clean up the locks by itself; it spends precision before it gains enough exploration at the fork. The improvement comes only when both stages act together. Training changes the distribution so the locks are safer; decoding then uses that extra room to explore the fork.

Real-model evidence. We now look for the same pattern from the base Qwen3-30B-Instruct model and its SSD counterpart on LCB v6. The same two signatures appear. Relative to the base model, SSD reaches decoding with a cleaner head and a weaker distractor tail.

[Figure 6a](#) shows the first effect directly. When tokens are ordered by probability, cumulative mass rises more quickly for SSD through the top ranks. Less probability is left behind in diffuse distractor tails before decoding even begins. This is the real-model analogue of the lock side of the toy.

[Figure 6b–d](#) show the second effect. Under the same evaluation-time decoding temperature and truncation $(T_{\text{eval}}, \rho_{\text{eval}})$, raising T_{eval} changes the base model much less: the surviving set stays close to a singleton, so temperature has limited leverage. SSD behaves differently. As temperature rises, several top continuations remain viable, and the probabilities among those surviving options spread out much more strongly. This advantage persists even when the two models place similar probability mass in their top 20 tokens. The real-model evidence therefore matches the